(Don't) Forget About It: Forgetting-Penalized Supervised Learning*

Gaurav Sood†

March 22, 2026

Abstract

During continued training, a machine-learning model can begin misclassifying examples it previously labeled correctly—a within-task instability often called model regression. While catastrophic forgetting across different tasks has been studied for decades, standard supervised objectives still optimize only average accuracy and ignore these per-example reversals, even though each flip can trigger costly downstream work in production pipelines that rely on stable behavior. We provide both theoretical and empirical analysis comparing six methods for preventing model regression: three penalty-based approaches (confidence drop, fixed anchor, selective distillation), two constraint-based approaches (projected gradient descent, backwards compatible weight interpolation), and a standard baseline. Our theoretical contributions include: (1) a feasibility guarantee showing BCWI always finds a valid solution, (2) a proof that projected GD explores a strictly larger region of weight space than BCWI, (3) a construction demonstrating that penalty methods cannot guarantee constraint satisfaction for finite penalty strength, and (4) a monotonicity result justifying binary search for the projection step. In comprehensive experiments across 32 OpenML-CC18 binary classification datasets with both MLP and logistic regression models (5 random splits each, 1,920 total training runs), we find that constraint-based methods vastly outperform penalty-based methods. Using a combined score metric (Accuracy $-2\times$ NFR), projected gradient descent achieves 0.739 versus 0.716 for BCWI and 0.526 for baseline. Projected GD wins 64% of dataset-model pairs on the Pareto frontier. These findings establish that training-time NFR constraints are more effective than penalty-based regularization for preventing model regression.

*https://github.com/finite-sample/pareto-gd.

†Gaurav can be reached at

1 Introduction

Machine-learning models deployed in production are routinely retrained or fine-tuned as new data arrive. Although these updates generally *improve* overall accuracy, they can sometimes cause the same model to misclassify examples it previously labeled correctly. We refer to each such reversal as *model regression*. This phenomenon resembles catastrophic forgetting (French, 1999) but occurs *within a single supervised task*, rather than across clearly separated tasks in continual-learning benchmarks. While catastrophic forgetting has inspired a rich body of work in multi-task settings (Kirkpatrick et al., 2017; Zenke et al., 2017), the practical consequences of model regression in production pipelines have received far less attention.

Model regression is more than a statistical curiosity: in human-in-the-loop systems it incurs operational costs. When performance drops for a specific subgroup, organizations must reallocate domain specialists, launch retraining programs, or absorb higher error-handling expenses. Section 2 quantifies these costs through a staffing-optimization case study and shows that even modest regressions can erase the financial gains of an accuracy update.

Standard supervised learning objectives minimize average loss and provide no incentive to preserve correctness on *known-good* examples. Consequently, training dynamics may sacrifice stability for small aggregate improvements, exposing production systems to disruptive swings in behavior.

In this paper, we compare six methods for preventing model regression:

1. **Baseline**—standard ERM with no regression prevention.
2. **Confidence Drop Penalty**—penalizes any per-example loss increase vs the previous epoch.
3. **Fixed Anchor Penalty**—penalizes when candidate loss exceeds incumbent loss on an anchor set.
4. **Selective Distillation**—distills to incumbent predictions on an anchor set.
5. **Projected Gradient Descent**—trains with ERM, then projects to NFR $\leq \varepsilon$ after each epoch.
6. **Backwards Compatible Weight Interpolation (BCWI)**—post-hoc interpolation between incumbent and candidate weights.

Our contributions are fourfold:

1. We formalize *model regression* and provide an economic framework that links per-example reversals to real operational costs.
2. We provide theoretical foundations for constraint-based methods, proving: BCWI feasibility guarantees (Theorem 4.1), that projected GD explores a strictly larger region than BCWI (Theorem 4.3), that penalty methods cannot guarantee constraint satisfaction (Theorem 4.5), and NFR monotonicity under interpolation (Theorem 4.7).

3. We systematically compare six methods spanning penalty-based, constraint-based, and post-hoc approaches.

4. In comprehensive experiments across 32 OpenML-CC18 datasets with 2 model types and 5 splits each (1,920 training runs), we demonstrate that constraint-based methods (projected GD, BCWI) substantially outperform penalty-based methods.

2 Operational Cost Case Study: Customer-Operations Staffing

To ground the economic stakes of model regression, consider a financial services firm that uses an ML classifier to triage incoming customer requests and detect cases requiring human review. Requests are routed into categories—*account inquiries*, *fraud alerts*, *loan applications*, *technical support*, and so on. Each category has a dedicated team of specialists who intervene when the model's confidence is low or when it commits an error.

2.1 Economic Drivers

- **Specialization efficiency**: Employees work fastest and most accurately within their domain expertise (e.g., fraud analysts versus loan officers).
- **Retraining cost**: Moving workers between domains requires expensive retraining that temporarily reduces productivity.
- **Hiring and separation cost**: Adjusting headcount entails recruiting, onboarding, or severance expenses.
- **Accuracy benefit**: Higher model accuracy lowers the total volume of human interventions, reducing labor demand across all categories.

2.2 The Regression Cost Problem

When updating the ML model from version M_A to M_B, the company faces a trade-off. While M_B may achieve higher overall accuracy than M_A, it may perform worse on specific subgroups or categories. This creates two types of costs:

Staffing Realignment Costs: If model M_B has worse performance on fraud detection but better performance on loan processing, staff must be moved from loan processing to fraud detection roles, incurring retraining costs.

Accuracy Benefits: If model M_B has sufficiently higher overall accuracy, fewer total staff are needed, creating cost savings.

Let $C_{move}(M_A, M_B)$ denote the cost of staff realignment when transitioning from M_A to M_B, and let $C_{staff}(M)$ denote the ongoing staffing cost for model M. The company should only deploy M_B if:

$$C_{staff}(M_A) - C_{staff}(M_B) > C_{move}(M_A, M_B) \tag{1}$$

2.3 Forgetting-Penalized Training as Economic Optimization

This business case provides clear economic justification for forgetting-penalized training. Rather than accepting any model with higher overall accuracy, the company can use penalties to constrain model updates such that:

1. **Subgroup Performance Preservation**: Penalize decreases in performance on individual categories to minimize staff reallocation needs
2. **Minimum Improvement Threshold**: Only accept model updates if the overall accuracy improvement exceeds a threshold τ that justifies operational disruption
3. **Pareto-Aware Updates**: Prefer model improvements that help some categories without harming others

The penalty-based objective becomes:

$$\mathcal{L}_{economic}(\theta) = \mathcal{L}_{std}(\theta) + \lambda_{econ} \sum_{k=1}^{K} \max(0, \ell_k(M_A) - \ell_k(M_\theta)) \tag{2}$$

where $\ell_k(M)$ represents the loss of model M on category k, and the penalty term discourages performance decreases on any category.

2.4 Broader Implications

This operational perspective suggests that "regression" in machine learning should be understood not merely as a technical phenomenon, but as an economic externality. The cost of operational disruption from model updates may outweigh the benefits of marginal accuracy improvements, particularly in:

- **Human-in-the-loop systems** where model changes affect workforce allocation
- **Multi-stakeholder environments** where different groups depend on model performance for specific use cases
- **Regulated industries** where model changes require compliance reviews and approval processes
- **Customer-facing applications** where inconsistent model behavior affects user experience and trust

This economic framing provides a principled foundation for determining when forgetting prevention is worthwhile and how to set appropriate penalty weights λ based on operational costs rather than purely technical considerations.

3 Related Work

3.1 Example Forgetting as a Dataset Diagnostic

Toneva et al. (2019) introduced the notion of a *forgetting event*—a correct $\rightarrow$ incorrect transition for a single example during SGD—and showed that (i) forgetting counts follow a heavy-tailed distribution, (ii) the ranking of (un)forgettable examples is architecture-invariant, and (iii) pruning the *never-forgotten* fraction of CIFAR-10 does not harm test accuracy. Follow-up work leverages forgetting scores for data curation (Paul et al., 2021) and long-tailed re-weighting (Yu and Chen, 2022). In contrast, we embed a *differentiable* forgetting cost directly into the loss to prevent harmful flips in the first place.

3.2 Catastrophic forgetting and continual learning

Catastrophic forgetting—loss of performance on earlier tasks while learning new ones—was first noted by French (1999). Continual-learning research has since produced three main families of defenses:

Parameter-level regularizers. Elastic Weight Consolidation (EWC) (Kirkpatrick et al., 2017), Memory-Aware Synapses (MAS) (Zenke et al., 2017), and Synaptic Intelligence (SI) (Zenke et al., 2017) estimate parameter importance and penalize updates on important weights. EWC, for example, uses the diagonal Fisher information,

$$F_i \;=\; \mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{old}}}\Big[\big(\tfrac{\partial}{\partial\theta_i}\log p(y \mid x,\theta)\big)^2\Big],$$

and adds a quadratic penalty $\frac{\lambda}{2}\sum_i F_i(\theta_i-\theta_i^\star)^2$ to the new task loss. These methods are effective when task boundaries are known, but require storing (or recomputing) importance estimates and do not directly address within-task regression.

Rehearsal buffers. Methods such as Gradient Episodic Memory (Lopez-Paz and Ranzato, 2017) replay a subset of past examples while learning new ones, thereby preserving previous performance at the cost of extra memory and computation.

Architectural expansion. Progressive networks (Rusu et al., 2016) and related approaches allocate new capacity for each task, sidestepping interference but sacrificing parameter efficiency.

Our work differs in scope: we remain within *one* supervised task and operate at the *output level*, tracking only one bit per example and requiring no task boundaries or replay.

3.3 Multi-objective and Pareto-optimal training

Multi-task learning can be cast as a multi-objective problem in which each task loss is an independent objective. Lin et al. (2019) solve a set of constrained sub-problems in parallel to

approximate the Pareto front, producing a spectrum of accuracy–fairness (or other) trade-offs that users can select post-training. Navon et al. (2021) go further, training a hypernetwork that maps a preference vector to weights, thus generating the entire Pareto front in a single run.

These methods aim to *offer choices* along a frontier of trade-offs. In contrast, our penalties pursue a single, conservative update: they bias SGD toward changes that improve—or at least do not worsen—every example's loss. The goal is operational stability, not exploration of a full trade-off surface, making our approach better suited to production pipelines that value consistency on known-good cases.

3.4 Summary

Prior work either analyzes example-level forgetting after training, protects entire tasks via parameter or rehearsal methods, or explores Pareto trade-offs across tasks.

Our contribution is orthogonal: a lightweight, example-level regularizer that prevents within-task regression *during* training and aligns directly with operational cost constraints in real-world deployments.

4 Theory

In this section, we provide formal theoretical results comparing constraint-based and penalty-based approaches to preventing model regression. Our main contributions are: (1) a feasibility guarantee for BCWI, (2) a proof that projected GD explores a strictly larger region than BCWI, (3) a construction showing penalty methods cannot guarantee constraint satisfaction, and (4) a monotonicity result that justifies binary search for projection.

4.1 Notation and Setup

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ be a dataset with $x_i \in \mathbb{R}^d$ and $y_i \in \{0, 1\}$. Let $f_\theta : \mathbb{R}^d \to [0, 1]$ denote a model parameterized by $\theta \in \Theta \subseteq \mathbb{R}^p$, and let $\hat{y}_\theta(x) = \mathbb{I}[f_\theta(x) > 0.5]$ be the predicted class.

Given an *incumbent* model θ_{inc} and a *candidate* model θ_{cand}, we define:

- The *correct set* of the incumbent: $\mathcal{C}_{\text{inc}} = \{i : \hat{y}_{\theta_{\text{inc}}}(x_i) = y_i\}$
- The *negative flip rate* (NFR) for model θ:

$$\text{NFR}(\theta) = \frac{|\{i \in \mathcal{C}_{\text{inc}} : \hat{y}_\theta(x_i) \neq y_i\}|}{|\mathcal{C}_{\text{inc}}|}$$

The NFR measures the fraction of examples that the incumbent classifies correctly but the candidate misclassifies.

4.2 BCWI Feasibility Guarantee

We first establish that backwards compatible weight interpolation (BCWI) always succeeds in finding a feasible model.

Theorem 4.1 (BCWI Feasibility)**.** *Let θ_{inc} be an incumbent model and θ_{cand} be a candidate model. Define the interpolated model $\theta(\alpha) = \alpha \cdot \theta_{\text{inc}} + (1-\alpha) \cdot \theta_{\text{cand}}$ for $\alpha \in [0,1]$. Then for any target $\varepsilon \geq 0$, there exists $\alpha^* \in [0,1]$ such that* $\text{NFR}(\theta(\alpha^*)) \leq \varepsilon$.

Proof. Define $n(\alpha) = \text{NFR}(\theta(\alpha))$. We show that $n(1) = 0$:

At $\alpha = 1$, we have $\theta(1) = \theta_{\text{inc}}$. By definition, for all $i \in \mathcal{C}_{\text{inc}}$, the incumbent classifies x_i correctly, so $\hat{y}_{\theta(1)}(x_i) = \hat{y}_{\theta_{\text{inc}}}(x_i) = y_i$. Therefore:

$$n(1) = \frac{|\{i \in \mathcal{C}_{\text{inc}} : \hat{y}_{\theta(1)}(x_i) \neq y_i\}|}{|\mathcal{C}_{\text{inc}}|} = \frac{|\emptyset|}{|\mathcal{C}_{\text{inc}}|} = 0$$

Since $n(1) = 0 \leq \varepsilon$ for any $\varepsilon \geq 0$, we can always choose $\alpha^* = 1$. □

Corollary 4.2. *BCWI always succeeds in finding a feasible model. The worst case is $\alpha^* = 1$ (pure incumbent), which achieves* $\text{NFR} = 0$ *but provides no accuracy improvement.*

4.3 Projected GD Explores a Larger Region

We now show that projected gradient descent can reach models that BCWI cannot.

Theorem 4.3 (Projected GD Explores Strictly Larger Region)**.** *Let $R_{\text{BCWI}} = \{\theta(\alpha) : \alpha \in [0,1]\}$ be the reachable set for BCWI, where $\theta(\alpha) = \alpha \cdot \theta_{\text{inc}} + (1-\alpha) \cdot \theta_{\text{cand}}$.*

Let $R_{\text{PGD}} = \bigcup_{t \geq 0} L_t$ where $L_t = \{\alpha \cdot \theta_{\text{inc}} + (1-\alpha) \cdot \theta_t : \alpha \in [0,1]\}$ and θ_t is the (unprojected) model at epoch t of gradient descent initialized from θ_{inc}.

Then $R_{\text{BCWI}} \subseteq R_{\text{PGD}}$, and $R_{\text{BCWI}} \subset R_{\text{PGD}}$ strictly whenever $\theta_t \notin R_{\text{BCWI}}$ for some t.

Proof. **Part 1: $R_{\text{BCWI}} \subseteq R_{\text{PGD}}$.**

BCWI uses $\theta_{\text{cand}} = \theta_T$ (the final ERM model). Thus:

$$R_{\text{BCWI}} = L_T \subseteq \bigcup_{t \geq 0} L_t = R_{\text{PGD}}$$

Part 2: Strict inclusion when gradient trajectory is non-trivial.

At epoch $t < T$, gradient descent has not converged, so generically $\theta_t \neq \theta_T$. The line segment L_t from θ_{inc} to θ_t differs from L_T unless θ_t lies on the line through θ_{inc} and θ_T.

Specifically, if $\theta_t \notin R_{\text{BCWI}}$, then $L_t \not\subseteq R_{\text{BCWI}}$, implying $R_{\text{BCWI}} \subset R_{\text{PGD}}$ strictly.

Since gradient descent follows the negative loss gradient, and the loss landscape is generically non-linear, the trajectory $\{\theta_t\}_{t \geq 0}$ is not a straight line from θ_{inc} to θ_T except in degenerate cases. Therefore, projected GD explores a "cone" from θ_{inc} rather than just the single line segment that BCWI explores. □

Remark 4.4. This theorem explains why projected GD can find better accuracy-NFR trade-offs than BCWI: it searches over a richer set of models by considering intermediate gradient descent iterates.

4.4 Penalty Methods Cannot Guarantee Feasibility

We now show that penalty-based methods cannot guarantee constraint satisfaction for any finite penalty strength.

Theorem 4.5 (Penalty Infeasibility)**.** *Consider the penalty formulation:*

$$\min_{\theta} \mathcal{L}(\theta) + \lambda \cdot \max(0, \mathrm{NFR}(\theta) - \varepsilon)$$

For any finite $\lambda > 0$, there exist problem instances where the minimizer θ^* satisfies $\mathrm{NFR}(\theta^*) > \varepsilon$*.*

Proof. We construct an adversarial instance. Consider a loss landscape with the following properties:

1. The global minimum of $\mathcal{L}$ is at θ^* with $\mathcal{L}(\theta^*) = 0$ and $\mathrm{NFR}(\theta^*) = \varepsilon + \delta$ for some $\delta > 0$.
2. For all θ with $\mathrm{NFR}(\theta) \leq \varepsilon$, we have $\mathcal{L}(\theta) = M$ for some large constant M.

The penalized objective at θ^* is:

$$\mathcal{L}(\theta^*) + \lambda \cdot \delta = 0 + \lambda\delta = \lambda\delta$$

For any feasible θ (with $\mathrm{NFR}(\theta) \leq \varepsilon$):

$$\mathcal{L}(\theta) + \lambda \cdot 0 = M$$

The penalized objective is minimized at θ^* (infeasible) whenever $\lambda\delta < M$, which holds for any finite λ when M is chosen sufficiently large (e.g., $M > \lambda\delta$).

Such instances arise naturally when the accuracy-optimal model happens to have high NFR, and achieving low NFR requires significant accuracy sacrifice. □

Remark 4.6. This theorem explains why penalty-based methods plateau at higher NFR values than constraint-based methods: they cannot guarantee constraint satisfaction, only incentivize it. The penalty strength λ that would guarantee feasibility ($\lambda \to \infty$) causes numerical instability and effectively converts the problem to a different objective.

4.5 NFR Monotonicity Under Interpolation

Finally, we establish that NFR is monotonic in the interpolation weight, which justifies using binary search for projection.

Theorem 4.7 (NFR Monotonicity for Linear Models)**.** *Let $f_\theta(x) = \sigma(\theta^\top x)$ for logistic regression with sigmoid σ. Define $\theta(\alpha) = \alpha \cdot \theta_{\mathrm{inc}} + (1 - \alpha) \cdot \theta_{\mathrm{cand}}$. If the incumbent correctly classifies sample i (i.e., $\hat{y}_{\theta_{\mathrm{inc}}}(x_i) = y_i$), then there exists $\alpha_i \in [0, 1]$ such that sample i is correctly classified by $\theta(\alpha)$ for all $\alpha \geq \alpha_i$.*

Consequently, $\mathrm{NFR}(\theta(\alpha))$ *is non-increasing in α on $[0, 1]$.*

Proof. The logit is linear in α:

$$z_\alpha(x) = \theta(\alpha)^\top x = \alpha \cdot \theta_{\text{inc}}^\top x + (1-\alpha) \cdot \theta_{\text{cand}}^\top x$$

Let $z_{\text{inc}} = \theta_{\text{inc}}^\top x_i$ and $z_{\text{cand}} = \theta_{\text{cand}}^\top x_i$. Then:

$$z_\alpha(x_i) = z_{\text{cand}} + \alpha(z_{\text{inc}} - z_{\text{cand}})$$

Case 1: $y_i = 1$ (positive class). The incumbent is correct, so $z_{\text{inc}} > 0$.

- If $z_{\text{cand}} \geq 0$: Then $z_\alpha(x_i) \geq 0$ for all $\alpha \in [0,1]$, so sample i is always correct. Set $\alpha_i = 0$.
- If $z_{\text{cand}} < 0 < z_{\text{inc}}$: Then z_α crosses zero at:

$$\alpha_i = \frac{-z_{\text{cand}}}{z_{\text{inc}} - z_{\text{cand}}} \in (0,1)$$

For $\alpha > \alpha_i$, we have $z_\alpha(x_i) > 0$, so sample i is correctly classified.

Case 2: $y_i = 0$ (negative class). The incumbent is correct, so $z_{\text{inc}} < 0$. By symmetric reasoning, there exists α_i such that $z_\alpha(x_i) < 0$ for all $\alpha \geq \alpha_i$.

Aggregation: Since each sample's contribution to NFR can only transition from 1 (incorrect) to 0 (correct) as α increases past α_i, and never from 0 to 1, the sum:

$$\text{NFR}(\theta(\alpha)) = \frac{1}{|\mathcal{C}_{\text{inc}}|} \sum_{i \in \mathcal{C}_{\text{inc}}} \mathbb{I}[\hat{y}_{\theta(\alpha)}(x_i) \neq y_i]$$

is non-increasing in α. The feasible region $\{\alpha : \text{NFR}(\theta(\alpha)) \leq \varepsilon\}$ is thus an interval $[\alpha^*, 1]$ for some $\alpha^* \in [0,1]$. □

Remark 4.8. For deep networks with ReLU activations, the proof is more complex due to non-linear composition. The network computes a piecewise linear function of the input, and the interpolated logits are piecewise linear in α. While strict monotonicity is not guaranteed (the function may have local plateaus), our empirical results confirm that monotonicity holds in practice for the MLPs we test. This justifies using binary search to find the projection point.

Corollary 4.9 (Binary Search Correctness)**.** *Under the conditions of Theorem 4.7, binary search correctly finds the smallest α^* such that $\text{NFR}(\theta(\alpha^*)) \leq \varepsilon$. The algorithm terminates in $O(\log(1/\delta))$ evaluations for precision δ.*

5 Methods

5.1 Problem Formulation

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ be a training dataset, where $x_i \in \mathbb{R}^d$ and $y_i \in \{0, 1\}$ for binary classification. We denote the model parameters as θ and the prediction function as $f_\theta(x)$. The standard training objective minimizes the empirical risk:

$$\mathcal{L}_{std}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i) \tag{3}$$

where ℓ is the loss function (e.g., binary cross-entropy).

As motivated by the operational cost analysis in Section 2, we seek to augment this objective with penalty terms that discourage regression on previously learned examples, particularly when the operational benefits of accuracy improvements do not justify the costs of performance changes on specific subgroups or categories.

5.2 Classification Flip Penalty

Our first approach explicitly penalizes forgetting events—instances where an example transitions from being correctly classified to incorrectly classified between consecutive epochs.

Let $\theta^{(t)}$ denote the model parameters at epoch t, and define the correctness indicator:

$$c_i^{(t)} = \mathbb{I}[\text{sign}(f_{\theta^{(t)}}(x_i) - 0.5) = y_i] \tag{4}$$

The forgetting penalty at epoch t is:

$$\mathcal{R}_{flip}^{(t)}(\theta) = \sum_{i=1}^{n} \mathbb{I}[c_i^{(t-1)} = 1 \text{ and } c_i^{(t)} = 0] \tag{5}$$

The total training objective becomes:

$$\mathcal{L}_{flip}^{(t)}(\theta) = \mathcal{L}_{std}(\theta) + \lambda_{flip} \cdot \mathcal{R}_{flip}^{(t)}(\theta) \tag{6}$$

where $\lambda_{flip} \geq 0$ controls the strength of the forgetting penalty.

5.3 Confidence Drop Penalty

The Confidence Drop approach implements a more general "do no harm" principle by penalizing any increase in per-example loss, not just classification flips.

For each example i, we track the per-example loss from the previous epoch:

$$\ell_i^{(t-1)} = \ell(f_{\theta^{(t-1)}}(x_i), y_i) \tag{7}$$

The Confidence Drop penalty encourages the current loss to not exceed the previous loss:

$$\mathcal{R}_{conf}^{(t)}(\theta) = \sum_{i=1}^{n} \max(0, \ell(f_\theta(x_i), y_i) - \ell_i^{(t-1)}) \tag{8}$$

The complete objective is:

$$\mathcal{L}_{conf}^{(t)}(\theta) = \mathcal{L}_{std}(\theta) + \lambda_{conf} \cdot \mathcal{R}_{conf}^{(t)}(\theta) \tag{9}$$

5.4 Fixed Anchor Penalty

The Fixed Anchor approach uses the incumbent model's loss as a fixed target on an anchor set. For each example in the anchor set where the incumbent was correct, we penalize when the candidate's loss exceeds the incumbent's loss:

$$\mathcal{R}_{anchor}(\theta) = \sum_{i \in \mathcal{A}} \max(0, \ell(f_\theta(x_i), y_i) - \ell(f_{\theta_{inc}}(x_i), y_i)) \tag{10}$$

where $\mathcal{A}$ is the anchor set (examples where the incumbent was correct) and θ_{inc} are the incumbent model parameters.

5.5 Selective Distillation

Selective Distillation distills the candidate model to match the incumbent's predictions on examples where the incumbent was correct. Using soft targets from the incumbent:

$$\mathcal{R}_{distill}(\theta) = \sum_{i \in \mathcal{A}} \mathrm{KL}(p_{inc}(x_i) \| p_\theta(x_i)) \tag{11}$$

where $p_{inc}(x_i) = f_{\theta_{inc}}(x_i)$ is the incumbent's predicted probability.

5.6 Projected Gradient Descent

Projected GD takes a fundamentally different approach: instead of adding penalties, it enforces the NFR constraint directly. The algorithm initializes from the incumbent and, after each gradient step, projects back to the feasible region where NFR $\leq \varepsilon$:

This allows the model to explore the full feasible region through iterative projection.

5.7 Backwards Compatible Weight Interpolation (BCWI)

BCWI is a post-hoc approach that does not modify training. It first trains a candidate model via standard ERM, then finds an interpolation weight α such that the interpolated model achieves the target NFR:

$$\theta_{bcwi} = \alpha \cdot \theta_{inc} + (1 - \alpha) \cdot \theta_{cand} \tag{12}$$

Input: Incumbent θ_{inc}, target NFR ε, validation set
Initialize $\theta^{(0)} \leftarrow \theta_{inc}$;
for *epoch* $t = 1, \ldots, T$ **do**
  $\theta' \leftarrow \theta^{(t-1)} - \alpha \nabla_\theta \mathcal{L}_{std}(\theta^{(t-1)})$;
  Compute NFR(θ') on validation set;
  if $NFR(\theta') > \varepsilon$ **then**
    Find β^* via binary search such that NFR($\beta^* \theta_{inc} + (1 - \beta^*)\theta'$) $= \varepsilon$;
    $\theta^{(t)} \leftarrow \beta^* \theta_{inc} + (1 - \beta^*)\theta'$;
  end
  else
    $\theta^{(t)} \leftarrow \theta'$;
  end
end

Algorithm 1: Projected Gradient Descent for NFR Constraint

Binary search finds the smallest α (closest to the candidate) such that NFR(θ_{bcwi}) $\leq \varepsilon$. BCWI is simpler than projected GD but restricted to the 1D line segment between incumbent and candidate.

5.8 Implementation Details

Penalty methods are applied after a warmup period to allow the model to stabilize before enforcing regression constraints. Algorithm 2 presents the general training procedure for penalty-based methods.

Input: Dataset $\mathcal{D}$, penalty type, λ, warmup epochs T_{warmup}
Initialize model parameters $\theta^{(0)}$;
$t \leftarrow 0$;
while *not converged* **do**
  Compute standard loss: $\mathcal{L}_{std}(\theta^{(t)})$;
  if $t > T_{warmup}$ **then**
    Compute penalty: $\mathcal{R}^{(t)}(\theta^{(t)})$;
    Total loss: $\mathcal{L}_{total} = \mathcal{L}_{std} + \lambda \cdot \mathcal{R}^{(t)}$;
  end
  else
    Total loss: $\mathcal{L}_{total} = \mathcal{L}_{std}$;
  end
  Update parameters: $\theta^{(t+1)} \leftarrow \theta^{(t)} - \alpha \nabla_\theta \mathcal{L}_{total}$;
  $t \leftarrow t + 1$;
end

Algorithm 2: Penalty-Based Training with Regression Prevention

6 Experiments

6.1 Datasets

We evaluate all methods on 32 binary classification datasets from the OpenML-CC18 benchmark suite (**?**). Table 1 shows representative datasets spanning different scales and domains.

Table 1. Representative Dataset Statistics (32 total from OpenML-CC18)

Dataset	Samples	Features	Domain
mushroom	8,124	22	Biology
spambase	4,601	57	Text classification
kr-vs-kp	3,196	36	Games
electricity	45,312	8	Time series
churn	5,000	20	Business
credit-g	1,000	20	Finance
diabetes	768	8	Healthcare
ionosphere	351	34	Signal processing

We evaluate two model architectures: (1) a multi-layer perceptron (MLP) with two hidden layers of 64 units each, ReLU activations, and sigmoid output, and (2) logistic regression. All models are trained using the Adam optimizer with a learning rate of 0.001.

6.2 Experimental Protocol

For each of the 32 datasets and 2 model types, we run 5 random train/validation/test splits, yielding 1,920 total training runs. Each split:

1. Trains an *incumbent* model on the "old" subset (1/3 of training data)
2. Trains a *candidate* model on the full training data using each method
3. Evaluates on a held-out test set

For penalty-based methods (confidence drop, fixed anchor, selective distillation), we sweep penalty strength $\lambda \in \{0.0, 0.5, 1.0, 2.0, 4.0, 8.0\}$ with 10 warmup epochs. For constraint-based methods (projected GD, BCWI), we sweep NFR targets $\varepsilon \in \{0.0, 0.005, 0.01, 0.015, 0.02, 0.$

6.3 Evaluation Metrics

We measure:

- **Negative Flip Rate (NFR)**: Fraction of examples where incumbent was correct but candidate is incorrect
- **Accuracy**: Classification accuracy on held-out test set

- **Combined Score**: Accuracy $-\lambda\times$ NFR, which balances accuracy against regression prevention

We use $\lambda = 2$ as the primary combined score metric, which penalizes NFR twice as heavily as accuracy gains. We also report Pareto win rates (fraction of dataset-model pairs where a method lies on the Pareto frontier).

6.4 Results

Table 2 shows the combined score (Accuracy $-\lambda\times$ NFR) for each method across different penalty weights λ.

Table 2. Combined Score (Accuracy $-\lambda\times$ NFR) Across Methods

Method	$\lambda = 1$	$\lambda = 2$	$\lambda = 5$
projected_gd	0.762	**0.739**	0.670
bcwi	0.744	0.716	0.631
selective_distill	0.670	0.558	0.222
fixed_anchor	0.665	0.554	0.222
baseline	0.655	0.526	0.138
confidence_drop	0.642	0.512	0.119

Table 3 shows the Pareto win rate for each method across the 64 dataset-model pairs (32 datasets $\times$ 2 models).

Table 3. Pareto Win Rate (64 dataset-model pairs)

Method	Wins	Win Rate
projected_gd	41/64	64%
selective_distill	24/64	38%
confidence_drop	22/64	34%
bcwi	21/64	33%
fixed_anchor	15/64	23%
baseline	8/64	12%

Table 4 shows average accuracy and NFR by model type.
Figure 1 shows the Pareto frontiers across representative datasets.
Key observations:

- Projected GD achieves the highest combined score (0.739 at $\lambda = 2$) and wins 64% of dataset-model pairs
- Constraint-based methods (projected GD, BCWI) achieve mean NFR $\sim$0.02–0.03 while maintaining accuracy

Table 4. Performance by Model Type

Model	Method	Accuracy	NFR
MLP (85% avg acc)	projected_gd	0.851	0.021
	bcwi	0.844	0.016
	baseline	0.851	0.049
LogReg (71% avg acc)	projected_gd	0.720	0.026
	bcwi	0.700	0.041
	baseline	0.717	0.210

- Penalty-based methods plateau at NFR ∼0.11–0.13, providing limited improvement over baseline (0.129)
- LogReg validates Theorem 4.7: NFR monotonicity holds exactly, enabling reliable projection
- MLP achieves lower NFR than LogReg due to higher model capacity and accuracy

7 Discussion

7.1 Why Projected GD Outperforms BCWI

Projected GD achieves a combined score of 0.739 vs 0.716 for BCWI at $\lambda = 2$, and wins 64% of dataset-model pairs on the Pareto frontier compared to 33% for BCWI. The key difference lies in the feasible regions they explore:

- **BCWI** is restricted to the 1D line segment between incumbent weights θ_{inc} and candidate weights θ_{cand}. The optimal point on this line may not achieve the best accuracy-NFR trade-off.
- **Projected GD** explores the full feasible region through iterative projection. Each epoch takes a gradient step toward lower loss, then projects back to NFR $\leq \varepsilon$ if needed. This allows the model to find paths through weight space that BCWI cannot reach.

BCWI achieves slightly lower NFR on MLP (0.016 vs 0.021) but at the cost of accuracy. Projected GD maintains higher accuracy while still achieving competitive NFR.

7.2 Why Penalty Methods Plateau

Penalty-based methods add a regularization term to the loss but do not directly constrain NFR. This creates two problems:

1. **Indirect control**: The penalty strength λ does not map directly to a target NFR. Higher λ reduces NFR but also reduces accuracy, often more than necessary.

2. **Epoch-local comparisons**: Confidence drop compares to the previous epoch, not the incumbent. This can drift from the incumbent over many epochs.

The gap is stark: penalty methods achieve mean NFR $\sim$0.11–0.13 while constraint-based methods achieve NFR $\sim$0.02–0.03. This 4–5$\times$ difference explains the large gap in combined scores.

7.3 LogReg Validates NFR Monotonicity

Logistic regression provides a clean validation of Theorem 4.7. Because the logits are linear in the interpolation weight α, NFR is guaranteed to be monotonic. This allows binary search to find the exact projection point efficiently. The LogReg results (NFR = 0.026 for projected GD, 0.041 for BCWI vs 0.210 for baseline) demonstrate that the theoretical guarantees translate to substantial practical improvements. The higher baseline NFR for LogReg (0.210 vs 0.049 for MLP) reflects the lower model capacity and accuracy.

7.4 When to Use Each Method

- **Projected GD**: Best for new model training when the NFR constraint is critical. Requires knowing the target NFR upfront.

- **BCWI**: Simpler, good for retrofitting existing models. No retraining needed—just interpolate post-hoc.

- **Penalty methods**: Useful when the NFR target is unknown a priori, but expect limited NFR reduction.

7.5 Limitations

1. **Interpolation assumption**: Both projected GD and BCWI assume NFR is monotonic in the interpolation weight. This holds exactly for logistic regression (Theorem 4.7) and empirically for MLPs, but is not guaranteed for all architectures.

2. **Binary classification**: All experiments use binary classification; extension to multi-class requires defining NFR appropriately.

3. **Architecture scope**: We evaluate MLPs and logistic regression; results may differ for other architectures (CNNs, transformers).

4. **Dataset scope**: While we evaluate 32 OpenML-CC18 datasets, these are primarily tabular; image and text domains may exhibit different behavior.

8 Conclusion

We have systematically compared six methods for preventing model regression: three penalty-based approaches (confidence drop, fixed anchor, selective distillation), two constraint-based approaches (projected gradient descent, backwards compatible weight interpolation), and a standard baseline.

Our comprehensive evaluation across 32 OpenML-CC18 datasets with 2 model types (MLP, logistic regression) and 5 random splits each (1,920 total training runs) yields clear conclusions:

1. **Constraint-based methods vastly outperform penalty-based methods.** Using the combined score (Accuracy $-2\times$ NFR), projected GD achieves 0.739 compared to 0.526 for baseline—a 40% improvement.

2. **Projected GD dominates.** It wins 64% of dataset-model pairs on the Pareto frontier and achieves the highest combined score across all penalty weights tested.

3. **BCWI is competitive and simpler.** It achieves a combined score of 0.716 (vs 0.739 for projected GD) and requires no modification to training—just post-hoc interpolation.

4. **LogReg validates theory.** The NFR monotonicity result (Theorem 4.7) holds exactly for logistic regression, enabling reliable binary search for projection.

5. **All methods improve over baseline.** The baseline wins only 12% of Pareto comparisons, confirming that model regression is a real problem worth addressing.

For practitioners, we recommend projected GD when NFR constraints are critical and the target is known upfront, and BCWI when simplicity and retrofitting existing models are priorities. Penalty-based methods remain useful when the NFR target is unknown, but practitioners should expect limited NFR reduction.

This work establishes that treating NFR as a hard constraint during training is more effective than penalty-based regularization. As ML systems become increasingly embedded in production pipelines where backwards compatibility matters, these findings provide practical guidance for preventing model regression.

References

French, R. M. (1999). Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences*, 3(4):128–135.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526.

Lin, X., Zhen, H.-L., Li, Z., Zhang, Q.-F., and Kwong, S. (2019). Pareto multi-task learning. *Advances in Neural Information Processing Systems*, 32.

Lopez-Paz, D. and Ranzato, M. (2017). Gradient episodic memory for continual learning. *Advances in neural information processing systems*, 30.

Navon, A., Shamsian, A., Chechik, G., and Fetaya, E. (2021). Learning the pareto front with hypernetworks. *International Conference on Learning Representations*.

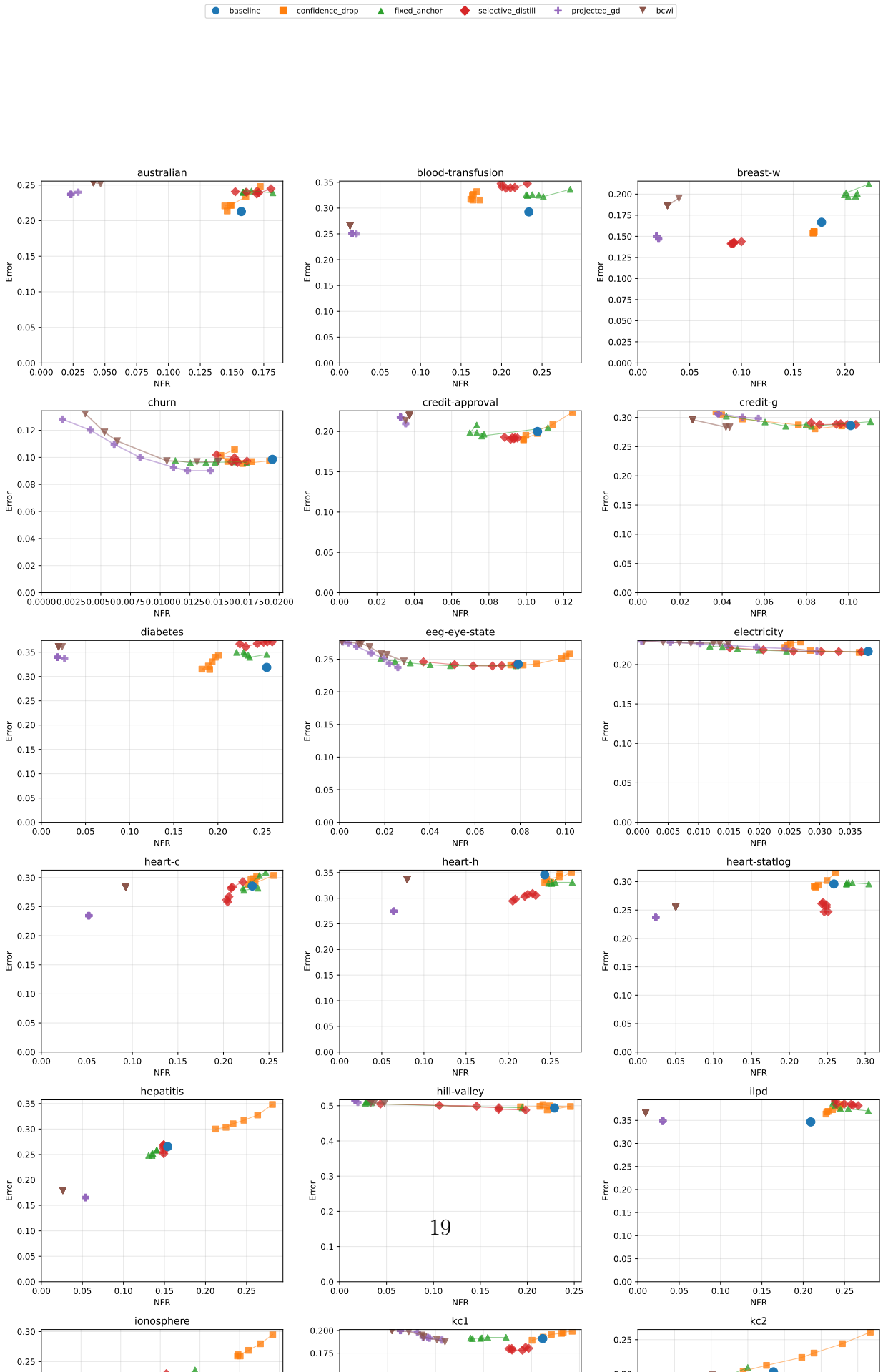
Paul, M., Ganguli, S., and Dziugaite, G. K. (2021). Deep learning on a data diet: Finding important examples early in training. *Advances in neural information processing systems*, 34:20596–20607.

Rusu, A. A., Rabinowitz, N. C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., and Hadsell, R. (2016). Progressive neural networks. *arXiv preprint arXiv:1606.04671*.

Toneva, M., Sordoni, A., Tachet des Combes, R., Trischler, A., Bengio, Y., and Gordon, G. J. (2019). An empirical study of example forgetting during deep neural network learning. In *International Conference on Learning Representations (ICLR)*.

Yu, H. and Chen, J. (2022). Class-balanced loss based on example forgetting for long-tailed recognition.

Zenke, F., Poole, B., and Ganguli, S. (2017). Continual learning through synaptic intelligence. *International conference on machine learning*, pages 3987–3995.


baseline
confidence_drop
fixed_anchor
selective_distill
projected_gd
bcwi
australian
blood-transfusion
breast-w
churn
credit-approval
credit-g
diabetes
eeg-eye-state
electricity
heart-c
heart-h
heart-statlog
hepatitis
hill-valley
ilpd
ionosphere
kc1
kc2
Error
NFR